

ELK + Kafka + Filebeat Complete Guide (Simple + Interview Ready)

Part 1: The Problem ELK Solves

Imagine this scenario in Microservices:

You have 10 microservices:

- └─ Order Service (3 instances)
- └─ Payment Service (2 instances)
- └─ Inventory Service (2 instances)
- └─ Notification Service (1 instance)
- └─ ... 6 more services

Total: ~20 application instances

Each writing logs to their OWN server:

Server 1: /var/log/order-service-1.log

Server 2: /var/log/order-service-2.log

Server 3: /var/log/payment-service-1.log

...

Server 20: /var/log/notification-service.log

User reports: "My order failed!"

Without ELK — what you have to do:

Step 1: SSH into order-service server 1 → grep logs 😬

Step 2: SSH into order-service server 2 → grep logs 😬

Step 3: SSH into payment-service server → grep logs 😬

Step 4: Manually correlate logs across servers 😬

Step 5: Try to reconstruct timeline 😬

Time taken: 30-60 minutes just to find the issue 😬

Meanwhile user is STILL facing the problem!

With ELK:

All logs from ALL services → sent to ELK in real-time

Developer opens Kibana (browser UI):

→ Search: "orderId=123" → instantly see ALL logs across ALL services ✅

→ Filter by time range → see exactly what happened ✅

→ See full request journey across services ✅

→ Visualize error rates on dashboard ✅

→ Get alerted when error rate spikes ✅

Time taken: 30 seconds 🚀

Part 2: What is ELK Stack?

ELK is an acronym for three open-source tools that work together:

E → Elasticsearch (Search & Storage engine)

L → Logstash (Data collection & processing pipeline)

K → Kibana (Visualization & UI)

Together they form a **centralized logging and analytics platform**.

Application Logs

↓

[Logstash] ← collect, parse, transform logs

↓

[Elasticsearch] ← store, index, make searchable

↓

[Kibana] ← visualize, search, dashboard, alert

Part3: Enterprise setup (with Kafka)

Microservices

↓

Filebeat / App logs

↓

Kafka (buffer + queue)

↓

Logstash (processing)

↓

Elasticsearch (storage/search)

↓

Kibana (dashboard)

Part 4: What each component REALLY does

Filebeat (log shipper)

- Reads log files
- Sends logs forward
- No heavy processing

 Think: “Courier delivery boy”

Kafka (buffer)

- Stores logs temporarily

- Handles spikes in traffic
- Ensures no data loss
- Allows replay of logs

👉 Think: “Traffic controller / safety buffer”

🔴 Logstash (processor 🧑)

- Parses logs
- Filters data
- Converts unstructured → structured logs

👉 Example:

ERROR payment failed userId=101

→ {level: ERROR, service: payment, userId: 101}

🟢 Elasticsearch (storage + search 🔍)

- Stores logs
- Provides fast search using inverted index
- Supports filtering and queries

👉 Think: “Google for logs”

🔵 Kibana (UI 📊)

- Visual dashboard
- Graphs, charts, error trends
- Log exploration

5 How data flows (step-by-step)

Step 1: Application generates logs

Payment service → ERROR log

Step 2: Filebeat collects logs

- Reads file continuously
- Sends forward

Step 3: Kafka stores logs (optional but powerful)

- Buffers logs safely
- Handles load spikes

Step 4: Logstash processes logs

- Extracts fields
- Cleans data

Step 5: Elasticsearch stores logs

- Indexes data for fast search

Step 6: Kibana displays logs

- Dashboards & graphs

6 Why Kafka is added (IMPORTANT INTERVIEW POINT)

Without Kafka:

- Logs go directly to Logstash/ELK
- If ELK fails → logs are lost ❌

With Kafka:

- Logs are stored safely
- System becomes reliable
- Supports high traffic

👉 Kafka = **Safety buffer + decoupling layer**

7 Filebeat vs Logstash (clear difference)

Feature	Filebeat	Logstash
Role	Ship logs	Process logs
Weight	Lightweight	Heavy
CPU usage	Low	High
Processing	Minimal	Advanced
Use case	Send logs	Transform logs

8 ELK vs Splunk (quick clarity)

Feature	ELK	Splunk
Type	Open-source	Paid product
Setup	Complex	Easy
Cost	Free	Expensive
Flexibility	High	Medium
UI	Kibana	Built-in Splunk UI

9. How LOG INDEXING works in Elasticsearch

Step 1: Log arrives in Elasticsearch

Example log:

```
{  
  "level": "ERROR",  
  "service": "payment",  
  "message": "Payment failed for user 101"  
}
```

Step 2: Elasticsearch creates an INDEX

👉 An index = collection of documents (like a database table)

Example:

index: logs-2026-06

Step 3: Text is ANALYZED (VERY IMPORTANT)

Elasticsearch breaks text into tokens:

Example:

"Payment failed for user 101"

Becomes:

["payment", "failed", "user", "101"]

👉 This is done using an analyzer

Step 4: Inverted Index is created

Instead of storing like a DB, Elasticsearch builds:

"payment" → doc1

"failed" → doc1

"user" → doc1

"101" → doc1

👉 This structure = INVERTED INDEX

✓ Word → Documents mapping

✗ Not Document → Words

10. How SEARCHING works in Elasticsearch

Step 1: User sends query

Example:

"payment failed user"

Step 2: Query is analyzed

["payment", "failed", "user"]

Step 3: Elasticsearch looks into inverted index

It finds:

payment → doc1

failed → doc1

user → doc1

Step 4: Scoring (BM25 algorithm)

Elasticsearch ranks results based on:

- How many words match
- How important words are
- How often they appear

 Best matching document gets highest score

Step 5: Results returned

Sorted results:

1. Most relevant logs
2. Less relevant logs